

CO4 – AT1 CODING CHALLENGE

Course Code: ITA0609

Course Title: Machine Learning

Student Name: S.Swagat

1. Write a Python program to implement the K-Nearest Neighbour(KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Sol:-

```
import math
```

```
# Robot Movement Dataset
```

```
# [Speed, Obstacle Distance, Class]
```

```
data = [  
    [10, 50, "Move"],  
    [15, 45, "Move"],  
    [20, 40, "Move"],  
    [40, 10, "Stop"],  
    [45, 8, "Stop"],  
    [50, 5, "Stop"]  
]
```

```
# Get K from user
```

```
k = int(input("Enter K: "))
```

```
# Get test data
```

```
x = float(input("Enter Speed: "))
```

```
y = float(input("Enter Obstacle Distance: "))
```

```
# Find distances
```

```
distances = []

for point in data:
    d = math.sqrt((point[0] - x) ** 2 + (point[1] - y) ** 2)
    distances.append([d, point[2]])

# Sort distances
distances.sort()

# Count classes of K nearest neighbours
move = 0
stop = 0

for i in range(k):
    if distances[i][1] == "Move":
        move += 1
    else:
        stop += 1

# Prediction
if move > stop:
    prediction = "Move"
else:
    prediction = "Stop"

print("Predicted Class:", prediction)
```

```
# Accuracy
```

```
correct = 0
```

```
output: Enter K: 3
```

```
Enter Speed: 18
```

```
Enter Obstacle Distance: 42
```

```
Predicted Class: Move
```

```
Accuracy: 50.0 %
```

```
for point in data:
```

```
    if point[2] == prediction:
```

```
        correct += 1
```

```
accuracy = (correct / len(data)) * 100
```

```
print("Accuracy:", accuracy, "%")
```

OUTPUT:-

```
Enter K: 3
Enter Speed: 18
Enter Obstacle Distance: 42
Predicted Class: Move
Accuracy: 50.0 %
```

2). Write a Python program to implement **Logistic Regression** for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

Sol:-

Program:-

```
import math

# [Signal Strength, Network Speed]

X = [
    [90, 50],
    [85, 45],
    [80, 40],
    [40, 15],
    [35, 10],
    [30, 8]
]

# 0 = Poor, 1 = Good
Y = [1, 1, 1, 0, 0, 0]

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + math.exp(-z))

# User enters learning rate
lr = float(input("Enter learning rate: "))

# Number of iterations
```

```
iterations = 1000
```

```
# Initial weights and bias
```

```
w1 = 0
```

```
w2 = 0
```

```
b = 0
```

```
# Gradient Descent
```

```
for i in range(iterations):
```

```
    dw1 = 0
```

```
    dw2 = 0
```

```
    db = 0
```

```
    for j in range(len(X)):
```

```
        z = w1 * X[j][0] + w2 * X[j][1] + b
```

```
        p = sigmoid(z)
```

```
        error = p - Y[j]
```

```
        dw1 += error * X[j][0]
```

```
        dw2 += error * X[j][1]
```

```
        db += error
```

```
# Update weights
```

```
w1 = w1 - lr * dw1 / len(X)
```

```
w2 = w2 - lr * dw2 / len(X)
```

```
b = b - lr * db / len(X)
```

```
# Display final weights
print("\nFinal weights:")
print("w1 =", round(w1, 4))
print("w2 =", round(w2, 4))
print("Bias =", round(b, 4))

# Test data
print("\nPrediction Results:")

correct = 0

for i in range(len(X)):

    
$$z = w1 * X[i][0] + w2 * X[i][1] + b$$


    probability = sigmoid(z)

    if probability >= 0.5:
        prediction = 1
    else:
        prediction = 0

    if prediction == 1:
        result = "Good"
    else:
        result = "Poor"

    if Y[i] == 1:
        actual = "Good"
    else:
```

```

        actual = "Poor"

    print("Input:", X[i],
          "Sigmoid:", round(probability, 3),
          "Actual:", actual,
          "Predicted:", result)

    if prediction == Y[i]:
        correct += 1

# Accuracy
accuracy = (correct / len(X)) * 100

print("\nAccuracy:", accuracy, "%")

```

OUTPUT:-

```

... Enter learning rate: 0.01

Final weights:
w1 = -0.4982
w2 = 1.1047
Bias = -0.1236

Prediction Results:
Input: [90, 50] Sigmoid: 1.0 Actual: Good Predicted: Good
Input: [85, 45] Sigmoid: 0.999 Actual: Good Predicted: Good
Input: [80, 40] Sigmoid: 0.985 Actual: Good Predicted: Good
Input: [40, 15] Sigmoid: 0.03 Actual: Poor Predicted: Poor
Input: [35, 10] Sigmoid: 0.001 Actual: Poor Predicted: Poor
Input: [30, 8] Sigmoid: 0.002 Actual: Poor Predicted: Poor

Accuracy: 100.0 %

```